

Link-Cut Trees

Nguồn gốc: Link-Cut Trees

`data-structure/link-cut-trees.pdf`

Link-Cut Tree là cấu trúc dữ liệu do Sleator và Tarjan đề xuất để giải các bài toán **cây động**. Với cấu trúc này, mỗi thao tác cơ bản trên cây động có thể thực hiện trong thời gian khấu hao $O(\log n)$.

Tài liệu này lần lượt giới thiệu khái niệm của Link-Cut Tree, các thao tác được hỗ trợ, cách viết Splay Tree bên trong Link-Cut Tree, rồi đi qua một số bài tập thực hành.

1 Định nghĩa Link-Cut Tree

Một vài thuật ngữ tiếng Anh thường gặp:

- **access**: truy cập;
- **preferred**: ưu tiên, được chọn;
- **path**: đường đi;
- **auxiliary**: phụ trợ.

Link-Cut Tree phân rã một cây thành nhiều chuỗi trên cây, rồi lưu riêng từng chuỗi trong một Splay Tree. Khi cần thao tác trên một đường đi trong cây gốc, ta dùng Splay để tách và ghép các chuỗi sao cho đường đi đó nằm trong cùng một Splay Tree; sau đó chỉ cần thao tác trên Splay Tree ấy.

Các khái niệm chính là:

- **Preferred Child**: con ưu tiên.
- **Preferred Edge**: cạnh ưu tiên.
- **Preferred Path**: đường ưu tiên.
- **Auxiliary Tree**: cây phụ trợ, tức Splay Tree lưu một đường ưu tiên.
- **Path Parent**: cha của đường.

Ví dụ, xét một cây gốc tại 1. Nếu con ưu tiên của 1 là 2, con ưu tiên của 2 là 4, còn 3, 4, 5, 6 không có con ưu tiên, thì đường $1 - 2 - 4$ là một đường ưu tiên. Cũng giống phân rã nặng nhẹ, Link-Cut Tree biến một cây thành nhiều chuỗi tuyến tính; điểm khác là mỗi chuỗi được lưu bằng Splay Tree để tiện tách và ghép động.

Trong Splay Tree của một đường ưu tiên, đỉnh gần gốc cây gốc hơn nằm về phía trái, đỉnh sâu hơn nằm về phía phải; nói cách khác, khóa sắp xếp là độ sâu deep. Gốc của Splay Tree có một con trở cha đặc biệt: đó là **path parent**, bằng cha trong cây gốc của đỉnh nông nhất trên đường này.

Nếu xem mỗi đường ưu tiên như một đỉnh và xem các cạnh không ưu tiên như cạnh giữa các đỉnh ấy, ta thu được một cây ảo. Cây ảo này thường nhỏ hơn cây ban đầu về cả chiều rộng lẫn chiều cao. Nếu cấu

trúc cây ảo không đổi, rồi trên mỗi đường ta dùng một cấu trúc tuyến tính để tăng tốc, ta nhận được ý tưởng của phân rã nặng nhẹ. Có thể hiểu phân rã nặng nhẹ là một Link-Cut Tree tĩnh. Tuy vậy, điểm cốt lõi của Link-Cut Tree không chỉ là nén cây, mà là khả năng đưa đường cần truy vấn thành một đường chính trong thời gian $O(\log n)$ khâu hao.

2 Các thao tác cơ bản

2.1 access

access là nền tảng của mọi thao tác trong Link-Cut Tree. Sau khi gọi `access(v)`, đường từ gốc của cây chứa v tới v trở thành một đường ưu tiên mới. Nói cách khác, tất cả các đỉnh trên đường từ gốc tới v trở thành con ưu tiên tương ứng, các cạnh trên đường ấy trở thành cạnh ưu tiên, và đường đó được lưu trong cùng một Splay Tree.

Cách làm là lặp trên các cây phụ trợ. Khi đang xử lý đỉnh x , trước hết `splay(x)` để đưa x lên gốc Splay Tree hiện tại. Vì đỉnh vừa được truy cập sẽ không giữ con ưu tiên cũ ở phía sâu hơn, ta cắt con phải của x . Sau đó nối phần đường đã xử lý trước đó vào con phải của x . Tiếp theo đi lên `parent` của Splay Tree hiện tại và lặp lại. Cuối cùng ta thu được một đường từ gốc cây gốc tới x .

Mô phỏng bằng lời cho `access(2)`: đưa 2 lên gốc của Splay Tree chứa nó và đặt con phải của 2 bằng 0; sau đó xử lý `parent`, ví dụ là 3, đưa 3 lên gốc, cắt con phải của 3 và nối đường chứa 2 vào đó. Nếu `parent` kế tiếp là 1, tiếp tục đưa 1 lên gốc, cắt con phải của 1 và nối đường chứa 3. Sau bước cuối, đường chính là đường từ gốc đến 2.

Lý do luôn thao tác với con phải là trong cây phụ trợ, phía phải chứa các đỉnh sâu hơn. Khi một đỉnh được đưa lên gốc Splay Tree, con trái của nó là đoạn đường về phía gốc cây gốc, còn con phải là đoạn đi xuống sâu hơn; ta cần thay đoạn đi xuống ấy bằng đường vừa truy cập.

```
void access(int root)
{
    int temp = 0;
    while (root)
    {
        splay(root);
        t[root].ch[1] = temp;
        updata(root);
        temp = root;
        root = t[root].fa;
    }
}
```

2.2 lca

Nếu không trực tiếp hỏi LCA thì thao tác này ít khi cần dùng; với cây tĩnh lại càng không cần dùng Link-Cut Tree. Nhưng nếu cây động hỏi trực tiếp LCA, ta có thể dựa vào access.

Giả sử cần tìm LCA của 9 và 6. Trước hết gọi `access(9)`, khi đó đường từ gốc tới 9 được dựng thành đường chính. Sau đó gọi `access(6)`. Hai đường từ gốc tới 9 và từ gốc tới 6 có một đoạn chung; đoạn chung này không cần thay đổi trong lần access thứ hai. Vì vậy, đỉnh cuối cùng bị thay đổi con phải chính là LCA. Trong Splay Tree, đỉnh cuối cùng được truy cập cũng vừa được xoay lên gốc, nên ta chỉ cần trả về biến tạm cuối cùng của access.

```
int access(int root)
```

```

{
    int temp = 0;
    while (root)
    {
        splay(root);
        t[root].ch[1] = temp;
        temp = root;
        root = t[root].fa;
    }
    return temp;
}

int lca(int root1, int root2)
{
    access(root1);
    return access(root2);
}

```

Trong ví dụ của tài liệu gốc, sau khi thực hiện `access(9)` rồi `access(6)`, đỉnh 5 là đỉnh cuối cùng bị đưa lên gốc cây phụ trợ, nên 5 là tổ tiên chung gần nhất của 6 và 9.

2.3 findroot

`findroot` tìm gốc của cây gốc, không phải gốc của Link-Cut Tree. Chính xác hơn, sau khi dựng đường chính, nó tìm đỉnh có độ sâu nhỏ nhất trên đường đó.

Sau `access(v)`, gốc cây gốc chắc chắn là đỉnh nhỏ nhất trong Auxiliary Tree chứa v . Ta đưa v lên gốc Splay Tree, rồi đi liên tục sang con trái cho đến khi không đi được nữa. Vì Splay Tree lưu đường theo thứ tự độ sâu, đỉnh ngoài cùng bên trái chính là gốc cây gốc.

```

int findroot(int root)
{
    access(root);
    splay(root);
    int temp = root;
    while (t[temp].ch[0]) temp = t[temp].ch[0];
    return temp;
}

```

Từ đây trở đi, ta xem Splay Tree như một công cụ; điều cần quan sát là thao tác vĩ mô trên các đường ưu tiên, không phải từng biến đổi nhỏ bên trong Splay.

2.4 makeroot hay changeroot

Nhiều thao tác cần đổi gốc của cây gốc. Trước hết dùng `access` để đặt gốc cũ và gốc mới vào cùng một đường chính. Khi đổi gốc từ 1 sang 6, các đường phụ khác không đổi; chỉ có đường chính bị đảo ngược. Vì thế `makeroot(x)` chỉ cần dựng đường từ gốc tới x , đưa x lên gốc cây phụ trợ, rồi đánh dấu lười đảo đoạn.

```

void changeroot(int root)
{
    access(root);

```

```

    splay(root);
    t[root].lazy ^= 1;
}

```

2.5 link

Theo định nghĩa, để nối v vào w , có thể truy cập v , đổi path parent của cây phụ trợ chứa v thành w , rồi truy cập lại v . Trong cài đặt, ta làm ngắn gọn hơn: biến `root1` thành gốc cây gốc, rồi đặt cha của nó là `root2`. Sau khi nối, gốc của cây hợp nhất là gốc của cây chứa `root2`; nếu cần một gốc khác, ta gọi `makeroot`.

```

void link(int root1, int root2)
{
    makeroot(root1);
    t[root1].fa = root2;
}

```

2.6 cut

Nếu `root1` và `root2` đang kề nhau trong cây, ta biến `root1` thành gốc, rồi `access(root2)`. Khi đó đường chính chỉ gồm hai đỉnh `root1` và `root2`; sau `splay(root2)`, `root1` là con trái của `root2`. Cắt cạnh bằng cách xóa con trái ấy và xóa cha của `root1`.

```

void cut(int root1, int root2)
{
    makeroot(root1);
    access(root2);
    splay(root2);
    t[root2].ch[0] = t[root1].fa = 0;
}

```

3 Cách viết Splay Tree trong Link-Cut Tree

Splay Tree dùng trong Link-Cut Tree hơi khác Splay Tree thông thường.

3.1 isroot

Trong Splay Tree độc lập, một đỉnh là gốc nếu cha của nó bằng 0. Trong Link-Cut Tree, gốc của một cây phụ trợ vẫn có thể trở cha tới path parent, tức một đỉnh thuộc cây phụ trợ khác. Vì cây phụ trợ kia không trở ngược lại bằng con trái hoặc con phải, ta kiểm tra một đỉnh có phải gốc Splay Tree hay không bằng cách xem cha của nó có nhận nó làm con hay không.

```

bool isroot(int root)
{
    return t[t[root].fa].ch[0] != root
        && t[t[root].fa].ch[1] != root;
}

```

3.2 Xoay

Khi xoay, cần chú ý ông của đỉnh có nằm trong cùng Splay Tree hay không. Nếu không, không được đặt con của ông thành đỉnh đang xoay; chỉ cần đặt cha của đỉnh đang xoay thành ông. So với Splay thường, khác biệt chính là kiểm tra `isroot(fa)`.

```
void rot(int root)
{
    int fa = t[root].fa;
    int gfa = t[fa].fa;
    int t1 = (root != t[fa].ch[0]);
    int t2 = (fa != t[gfa].ch[0]);
    int ch = t[root].ch[1 ^ t1];

    if (!isroot(fa)) t[gfa].ch[0 ^ t2] = root;

    t[ch].fa = fa;
    t[root].fa = gfa;
    t[root].ch[1 ^ t1] = fa;
    t[fa].fa = root;
    t[fa].ch[0 ^ t1] = ch;
    /* If extra data is maintained, update(fa) here. */
}
```

3.3 splay

Khi splay, thứ nhất không được xoay sang cây phụ trợ khác. Thứ hai, các đánh dấu lười phải được đẩy xuống theo đúng thứ tự từ trên xuống dưới. Ta đi từ đỉnh hiện tại lên gốc Splay Tree, lưu đường đi vào một ngăn xếp, rồi pushdown ngược lại.

```
void pushdown(int root)
{
    if (t[root].lazy)
    {
        t[root].lazy = 0;
        swap(t[root].ch[0], t[root].ch[1]);
        t[t[root].ch[0]].lazy ^= 1;
        t[t[root].ch[1]].lazy ^= 1;
    }
}

void splay(int root)
{
    stk[++top] = root;
    for (int i = root; !isroot(i); i = t[i].fa)
        stk[++top] = t[i].fa;

    while (top) pushdown(stk[top--]);

    while (!isroot(root))
    {
        int fa = t[root].fa;
        int gfa = t[fa].fa;
        if (!isroot(fa))
```

```

        root == t[fa].ch[0] ^ fa == t[gfa].ch[0]
        ? rot(root) : rot(fa);
    rot(root);
}
}

```

4 Bài tập thực hành

4.1 Duy trì động cấu trúc cây: Cave Survey

Bài *Cave Survey* (codevs1839) cho n hang động và m lệnh. Ban đầu không có đường hầm. Mỗi lệnh là:

- Connect u v : xuất hiện một đường hầm giữa u và v ;
- Destroy u v : đường hầm giữa u và v bị phá;
- Query u v : hỏi u và v có liên thông hay không.

Đề bảo đảm tại mọi thời điểm giữa hai hang bất kỳ có nhiều nhất một đường đi, tức đồ thị luôn là một rừng. Vì có xóa cạnh nên DSU thường không dùng trực tiếp được. Với Link-Cut Tree, kiểm tra liên thông bằng cách so sánh gốc của hai đỉnh: nếu $\text{findroot}(u) == \text{findroot}(v)$ thì liên thông.

Dữ liệu: $n \leq 10000$, $m \leq 200000$. Mọi lệnh Destroy đều phá một cạnh đang tồn tại. Do dữ liệu lớn, cài đặt C/C++ nên dùng scanf và printf.

```

const int MAXN = 100005;
struct tree { int fa, ch[2], lazy; } t[MAXN];
int stk[MAXN], top;

void pushdown(int root) {
    if (t[root].lazy) {
        t[root].lazy = 0;
        swap(t[root].ch[0], t[root].ch[1]);
        t[t[root].ch[0]].lazy ^= 1;
        t[t[root].ch[1]].lazy ^= 1;
    }
}

bool isroot(int root) {
    return t[t[root].fa].ch[0] != root
        && t[t[root].fa].ch[1] != root;
}

void rot(int root) {
    int fa = t[root].fa, gfa = t[fa].fa;
    int t1 = (root != t[fa].ch[0]);
    int t2 = (fa != t[gfa].ch[0]);
    int ch = t[root].ch[1 ^ t1];
    if (!isroot(fa)) t[gfa].ch[0 ^ t2] = root;
    t[ch].fa = fa;
    t[root].fa = gfa;
    t[root].ch[1 ^ t1] = fa;
    t[fa].fa = root;
    t[fa].ch[0 ^ t1] = ch;
}

```

```

void splay(int root) {
    stk[++top] = root;
    for (int i = root; !isroot(i); i = t[i].fa)
        stk[++top] = t[i].fa;
    while (top) pushdown(stk[top--]);
    while (!isroot(root)) {
        int fa = t[root].fa, gfa = t[fa].fa;
        if (!isroot(fa))
            root == t[fa].ch[0] ^ fa == t[gfa].ch[0]
                ? rot(root) : rot(fa);
        rot(root);
    }
}

void access(int root) {
    int temp = 0;
    while (root) {
        splay(root);
        t[root].ch[1] = temp;
        temp = root;
        root = t[root].fa;
    }
}

void makeroot(int root) {
    access(root);
    splay(root);
    t[root].lazy ^= 1;
}

void link(int root1, int root2) {
    makeroot(root1);
    t[root1].fa = root2;
}

void cut(int root1, int root2) {
    makeroot(root1);
    access(root2);
    splay(root2);
    t[root2].ch[0] = t[root1].fa = 0;
}

int findroot(int root) {
    access(root);
    splay(root);
    int temp = root;
    while (t[temp].ch[0]) temp = t[temp].ch[0];
    return temp;
}

/* main: read commands Q/C/D.
   Q: print Yes if findroot(a) == findroot(b), otherwise No.
   C: link(a,b).

```

```
D: cut(a,b). */
```

4.2 Duy trì bài toán đường đi đơn giản: Bouncing Sheep

Bài *Bouncing Sheep* (codevs2333) có n thiết bị bật, đánh số 0 đến $n - 1$. Tại vị trí i , thiết bị đẩy con cừu tới $i + k_i$; nếu vượt ra ngoài thì con cừu bay ra khỏi dãy. Hai thao tác:

- hỏi từ thiết bị j cần bao nhiêu lần bật để bay ra;
- đổi hệ số bật của thiết bị j thành k .

Thêm một đỉnh giả $n + 1$ biểu diễn trạng thái đã bay ra. Mỗi thiết bị có đúng một cạnh đi tới thiết bị kế tiếp hoặc đỉnh giả; vì thế ta có một rừng động. Đáp án truy vấn là số đỉnh trên đường từ j tới $n + 1$ trừ 1. Trong Splay Tree cần duy trì kích thước đoạn đường.

```
struct tree { int fa, ch[2], size, lazy; } t[MAXN];
int nxt[MAXN];

int nexts(int x, int y) {
    if (x + y > n) return n + 1;
    return x + y;
}

void updata(int root) {
    t[root].size = t[t[root].ch[0]].size
        + t[t[root].ch[1]].size + 1;
}

void work(int root1, int root2) {
    makeroot(root1);
    access(root2);
    splay(root2);
}

/* Init: for every i, nxt[i] = nexts(i, k_i), link(i, nxt[i]).
   Type-1 query at a:
       ++a; work(a, n + 1); print t[n + 1].size - 1.
   Type-2 update at a with b:
       ++a; temp = nexts(a, b);
       if temp != nxt[a]: cut(a, nxt[a]); nxt[a] = temp; link(a, nxt[a]).
*/
```

Tài liệu gốc nhấn mạnh một tối ưu nhỏ: giảm số lần gọi update có thể tăng tốc đáng kể ở những bài phải duy trì nhiều thông tin trên Splay Tree. Với bài này hiệu quả không quá rõ, nhưng ở các bài nặng hơn thì đáng kể.

Các lỗi thường gặp:

1. Vòng lặp vô hạn: phần lớn do viết sai splay. Khi gỡ lỗi, in root, cha và ông của nó trong mỗi bước.
2. Kết quả sai: sau work, dùng hàm debug để kiểm tra root1 có thật là gốc không, và đường từ root2 tới root1 có thật là đường chính không. Nếu cấu trúc sai, kiểm tra access. Nếu cấu trúc đúng, kiểm tra đánh dấu lười trên đường và hàm update.

4.3 Trọng số cạnh hay trọng số đỉnh

Bài *Small Computer Room Tree* (codevs2370) cho một cây N đỉnh đánh số từ 0 đến $N - 1$, mỗi cạnh có chi phí c . Với mỗi truy vấn hai đỉnh u, v , cần in độ dài đường đi ngắn nhất giữa chúng. Đây là bài LCA khá trần, nhưng dùng để minh họa cách Link-Cut Tree xử lý trọng số cạnh.

Link-Cut Tree thuận tiện duy trì thông tin trên đỉnh, không trực tiếp trên cạnh. Cách chuẩn là biến mỗi cạnh thành một đỉnh mới mang trọng số của cạnh đó, rồi nối u với đỉnh cạnh và đỉnh cạnh với v . Khi hỏi đường đi, duyệt đường từ u tới v và lấy tổng trên Splay Tree.

```
const int MAXN = 225005;
struct tree { int fa, ch[2], num, sum, lazy; } t[MAXN];

void updata(int root) {
    t[root].sum = t[t[root].ch[0]].sum
        + t[t[root].ch[1]].sum
        + t[root].num;
}

void work(int root1, int root2) {
    changeroot(root1);
    access(root2);
    splay(root2);
}

int main() {
    scanf("%d", &n);
    for (int i = 1; i < n; ++i) {
        scanf("%d %d %d", &a1, &b1, &t[n+i].num);
        ++a1; ++b1;
        t[n+i].sum = t[n+i].num;
        link(a1, n+i);
        link(n+i, b1);
    }
    scanf("%d", &m);
    for (int i = 1; i <= m; ++i) {
        scanf("%d %d", &a1, &b1);
        ++a1; ++b1;
        work(a1, b1);
        printf("%d\n", t[b1].sum);
    }
}
```

4.4 Cây tĩnh với thông tin đoạn phức tạp hơn

Bài *Tree Statistics* (codevs2460, tuyển chọn đội tuyển tỉnh Chiết Giang 2008) cho cây n đỉnh, mỗi đỉnh có trọng số w . Các thao tác:

- CHANGE u t : đổi trọng số đỉnh u thành t ;
- QMAX u v : hỏi trọng số lớn nhất trên đường $u-v$;
- QSUM u v : hỏi tổng trọng số trên đường $u-v$.

Dữ liệu: $n \leq 30000$, $q \leq 200000$, trọng số nằm trong đoạn $[-30000, 30000]$.

Điểm cần giải thích là sửa một đỉnh: `splay(u)` để đưa đỉnh đó lên gốc cây phụ trợ, sửa trực tiếp `num`, rồi `update`. Với truy vấn đường đi, dùng `makeroot(u)`; `access(v)`; `splay(v)` rồi đọc thông tin ở `v`.

```
struct tree { int ch[2], num, fa, lazy, maxx, sums; } t[MAXN];

void updata(int root) {
    t[root].maxx = max(t[root].num,
        max(t[t[root].ch[0]].maxx, t[t[root].ch[1]].maxx));
    t[root].sums = t[t[root].ch[0]].sums
        + t[t[root].ch[1]].sums
        + t[root].num;
}

void work(int root1, int root2) {
    makeroot(root1);
    access(root2);
    splay(root2);
}

/* After linking the initial edges:
   t[0].sums = 0; t[0].maxx = -INF.
   CHANGE a b: splay(a); t[a].num = b; updata(a).
   QMAX a b: work(a,b); in t[b].maxx.
   QSUM a b: work(a,b); in t[b].sums.
*/
```

4.5 Dùng nhiều đánh dấu lười và update phức tạp

Bài *Coloring* (codevs1566, tuyển chọn đội tuyển tỉnh Sơn Đông) cho cây vô hướng n đỉnh và m thao tác:

- C a b c: tô toàn bộ các đỉnh trên đường $a-b$ thành màu c ;
- Q a b: hỏi số đoạn màu trên đường $a-b$, trong đó các đỉnh liên tiếp cùng màu được xem là một đoạn.

Bài này có sửa đoạn, nên cần đánh dấu lười. Đồng thời tồn tại hai loại đánh dấu: đánh dấu đảo chiều và đánh dấu gán màu. Hai loại này không cản trở nhau trong bài này. Nếu là đánh dấu gán và đánh dấu cộng thì chúng có thể ảnh hưởng lẫn nhau, khi đó phải xử lý thứ tự cẩn thận hơn.

Khi tính `update`, nếu con còn đánh dấu lười chưa đẩy xuống thì kết quả có thể sai, tương tự truy vấn cực trị trong cây đoạn. Vì vậy trước khi tổng hợp, mã nguồn đẩy đánh dấu xuống hai con. Nếu mọi giá trị nhỏ nhất trả về thành 0, cần kiểm tra đỉnh ảo 0 đã được gán giá trị biên phù hợp chưa.

```
struct tree {
    int ch[2], fa, lx, rx, num, col, lazy1, lazy2;
} t[MAXN];

void pushdown(int root) {
    if (t[root].lazy1) {
        swap(t[root].ch[0], t[root].ch[1]);
        swap(t[root].lx, t[root].rx);
        t[root].lazy1 = 0;
        t[t[root].ch[0]].lazy1 ^= 1;
        t[t[root].ch[1]].lazy1 ^= 1;
    }
}
```

```

    if (t[root].lazy2) {
        int c = t[root].lazy2;
        t[t[root].ch[0]].lazy2 = c;
        t[t[root].ch[1]].lazy2 = c;
        t[root].lx = t[root].rx = t[root].col = c;
        t[root].num = 1;
        t[root].lazy2 = 0;
    }
}

void updata(int root) {
    if (t[root].ch[0]) pushdown(t[root].ch[0]);
    if (t[root].ch[1]) pushdown(t[root].ch[1]);
    t[root].num = t[t[root].ch[0]].num
        + t[t[root].ch[1]].num + 1;
    if (t[t[root].ch[0]].rx == t[root].col) --t[root].num;
    if (t[t[root].ch[1]].lx == t[root].col) --t[root].num;
    t[root].lx = t[root].ch[0] ? t[t[root].ch[0]].lx : t[root].col;
    t[root].rx = t[root].ch[1] ? t[t[root].ch[1]].rx : t[root].col;
}

void change(int root1, int root2, int cols) {
    makeroot(root1);
    access(root2);
    splay(root2);
    t[root2].lazy2 = cols;
}

void work(int root1, int root2) {
    makeroot(root1);
    access(root2);
    splay(root2);
}

/* Q a b: work(a,b); in t[b].num.
   C a b c: change(a,b,c). */

```

Tác giả gốc nhận xét rằng bài này khó viết đúng ngay lần đầu; nó thích hợp để luyện kỹ năng gỡ lỗi Link-Cut Tree.

4.6 Fruit Street III

Bài *Fruit Street III* (codevs3306) cũng minh họa tác động của đánh dấu đảo chiều lên thông tin cần duy trì. Có n cửa hàng trên một cây, mỗi cửa hàng có giá táo. Hai thao tác:

- $0 \ x \ y$: đổi giá cửa hàng x thành y ;
- $1 \ x \ y$: trên đường từ x tới y , chọn một cửa hàng để mua và một cửa hàng về sau trên đường để bán, hỏi lợi nhuận lớn nhất.

Cần duy trì trên mỗi Splay Tree:

- maxx: giá lớn nhất;
- mins: giá nhỏ nhất;

- ans1: lợi nhuận tốt nhất theo hướng trái sang phải;
- ans2: lợi nhuận tốt nhất theo hướng phải sang trái.

Khi đảo đoạn, hai con bị đổi chỗ và hai đáp án có hướng ans1, ans2 cũng phải đổi chỗ.

```
struct tree {
    int ch[2], fa, maxx, mins, num, ans1, ans2, lazy;
} t[MAXN];

void pushdown(int root) {
    if (t[root].lazy) {
        t[root].lazy = 0;
        swap(t[root].ch[0], t[root].ch[1]);
        swap(t[root].ans1, t[root].ans2);
        t[t[root].ch[0]].lazy ^= 1;
        t[t[root].ch[1]].lazy ^= 1;
    }
}

void update(int root) {
    if (t[root].ch[0]) pushdown(t[root].ch[0]);
    if (t[root].ch[1]) pushdown(t[root].ch[1]);
    int l = t[root].ch[0], r = t[root].ch[1];
    t[root].maxx = max(t[root].num, max(t[l].maxx, t[r].maxx));
    t[root].mins = min(t[root].num, min(t[l].mins, t[r].mins));
    t[root].ans1 = max(max(t[r].maxx, t[root].num)
        - min(t[l].mins, t[root].num),
        max(t[l].ans1, t[r].ans1));
    t[root].ans2 = max(max(t[l].maxx, t[root].num)
        - min(t[r].mins, t[root].num),
        max(t[l].ans2, t[r].ans2));
}

void change(int root, int d) {
    splay(root);
    t[root].num = d;
    update(root);
}

/* Query 1 x y: work(x,y); print t[y].ans1. */
```

4.7 Cây khung nhỏ nhất động: Comfortable Route

Bài *Comfortable Route* (codevs1001) cho N điểm du lịch và M con đường hai chiều, mỗi đường có tốc độ bắt buộc v_i . Với hai điểm s, t , cần tìm một đường đi sao cho tỉ số giữa tốc độ lớn nhất và nhỏ nhất trên đường là nhỏ nhất; nếu không có đường đi thì in IMPOSSIBLE, nếu cần thì in phân số tối giản.

Cách DSU đơn giản

Sắp xếp cạnh theo trọng số tăng dần. Lần lượt chọn mỗi cạnh làm ans_max; sau đó duyệt ngược các cạnh không lớn hơn nó và dùng DSU để nối các thành phần liên thông. Khi s và t lần đầu liên thông, cạnh hiện tại chính là ans_min lớn nhất có thể ứng với ans_max này. Cập nhật tỉ số tốt nhất.

Nguyên lý là tham lam theo ans_min : khi đã cố định tốc độ lớn nhất, tốc độ nhỏ nhất càng lớn càng tốt. Độ phức tạp theo mã gốc là $O(n^2 \log n)$, bộ nhớ $O(n)$.

```
sort(edge + 1, edge + 1 + m, by_weight);
for (int i = 1; i <= m; ++i) {
    reset_dsu();
    for (int j = i; j >= 1; --j) {
        unite(edge[j].u, edge[j].v);
        if (find(s) == find(t)) {
            relax(edge[i].w, edge[j].w);
            break;
        }
    }
}
```

Cách SPFA

Một hướng khác là cố định giới hạn lớn nhất max . Với các cạnh có trọng số không vượt quá max , dùng SPFA để tối đa hóa giá trị nhỏ nhất trên đường từ s tới mọi đỉnh. Nếu duyệt max tăng dần, mỗi lần chỉ cần đưa hai đầu của cạnh mới vào hàng đợi và không cần xóa mảng dis . Khi $dis[t]$ tăng, ans_max là trọng số đang duyệt và ans_min là $dis[t]$.

```
int spfa(int maxvi)
{
    while (head != tail)
    {
        int x = pop_queue();
        for (edge i from x)
            if (i.vi <= maxvi
                && min(dmin[x], i.vi) > dmin[i.to])
            {
                dmin[i.to] = min(dmin[x], i.vi);
                push(i.to);
            }
        if (dmin[t] changed) relax(maxvi, dmin[t]);
    }
}
```

Cách Link-Cut Tree

Cách dùng Link-Cut Tree tương tự bài *Magic Forest*. Duyệt các cạnh theo trọng số tăng dần, coi cạnh đang xét là ứng viên cho trọng số lớn nhất. Ta duy trì một cây khung cực đại theo các cạnh đã xét: trên đường giữa hai đỉnh bất kỳ, cạnh nhỏ nhất trên cây khung này là lớn nhất có thể.

Khi thêm cạnh (u, v, w) :

- nếu u và v chưa liên thông, thêm cạnh vào cây;
- nếu đã liên thông, cạnh mới tạo một chu trình. Tìm cạnh có trọng số nhỏ nhất trên đường $u-v$; nếu nó nhỏ hơn w , cắt cạnh nhỏ nhất ấy và thêm cạnh mới, ngược lại bỏ qua cạnh mới.

Sau mỗi bước, nếu s và t liên thông, dựng đường $s-t$, lấy trọng số nhỏ nhất trên đường làm ans_min , còn cạnh đang duyệt làm ans_max , rồi cập nhật đáp án. Độ phức tạp khâu hao là $O(m \log n)$, bộ nhớ $O(n + m)$.

Như ở phần trọng số cạnh, mỗi cạnh được biểu diễn bằng một đỉnh mới mang trọng số cạnh; các đỉnh gốc nhận trọng số vô cùng lớn để không ảnh hưởng tới phép lấy min.

```
struct tree { int fa, ch[2], num, mins, lazy; } t[MAXN];

void update(int root) {
    t[root].mins = root;
    if (t[t[root].ch[0]].mins.num < t[t[root].mins].num)
        t[root].mins = t[t[root].ch[0]].mins;
    if (t[t[t[root].ch[1]].mins].num < t[t[root].mins].num)
        t[root].mins = t[t[t[root].ch[1]].mins];
}

for (int i = 1; i <= m; ++i) {
    int u = e[i].p[0], v = e[i].p[1], w = e[i].vi;
    if (findroot(u) != findroot(v)) {
        t[i+n].num = w;
        t[i+n].mins = i+n;
        t[u].num = t[v].num = INF;
        link(u, i+n);
        link(i+n, v);
    } else {
        works(u, v);
        int old = t[v].mins;
        if (t[old].num < w) {
            cut(e[old-n].p[0], old);
            cut(old, e[old-n].p[1]);
            t[i+n].num = w;
            t[i+n].mins = i+n;
            link(u, i+n);
            link(i+n, v);
        }
    }
}

if (findroot(s) == findroot(t)) {
    works(s, t);
    relax(w, t[t[t].mins].num); /* idea: min edge on path s--t */
}
}
```

Dòng cuối trong bản trích trên chỉ diễn đạt ý tưởng của mã nguồn gốc: sau `works(s, t)`, thông tin min nằm ở gốc Splay Tree của đường $s-t$. Trong mã hoàn chỉnh, biến đích là `t1` và biểu thức tương ứng là trọng số của đỉnh cạnh nhỏ nhất trên đường ấy.

Tác giả gốc kết luận bằng gợi ý: hãy dùng cả SPFA và Link-Cut Tree để giải *NOI Magic Forest*; bài đó gần như cùng khuôn mẫu với lời giải Link-Cut Tree ở trên.

5 Ghi chú về độ phức tạp

Các thao tác cốt lõi `access`, `makeroot`, `link`, `cut`, `findroot` đều được xây trên Splay Tree. Mỗi lần `access` có thể thay đổi nhiều cạnh ưu tiên, nhưng phân tích khấu hao của Sleator–Tarjan cho thấy tổng chi phí được chặn bởi $O(\log n)$ cho mỗi thao tác. Vì vậy, trong các bài cây động, mỗi cập nhật hoặc truy

vấn đường đi thường đạt $O(\log n)$ khấu hao, cộng thêm chi phí duy trì thông tin đoạn trong update và pushdown.